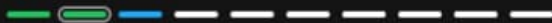


QUESTION 2 OF 10



Preview only — answers can't be changed or retried here.

What is a **Docker Registry** ?

It is a storage and distribution system for named Docker images



All of the options

It is a place to store Dockerfiles

It is an alternative to code hosting platform for version control like Github, GitLab etc.

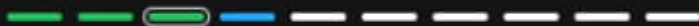
[BACK TO CURRENT QUESTION](#)

[← Previous](#)

[Next →](#)

~ → 

QUESTION 3 OF 10



Preview only — answers can't be changed or retried here.

By default, the `Docker engine` interacts with ?

DockerHub



||

QUESTION 5 OF 10



Preview only — answers can't be changed or retried here.

Which command is used for `login` to a `self-hosted registry`?

`docker connect [SERVER]`

`docker load [SERVER]`

`docker login [SERVER]` 

`docker sign-in [SERVER]`



Correct Answer

~ → 

QUESTION 6 OF 10



Let practice deploying a `registry server` on our own.
Run a registry server with name equals to `my-registry` using `registry:2` image with host port set to `5000`, and restart policy set to `always`.

Note: Registry server is exposed on port `5000` in the image.

Here we are hosting our own registry using the open source Docker Registry.

NOTE: You do not need to run docker login for this local registry setup because authentication is not configured. This registry allows for anonymous access.

[Check](#)

```
~ → docker run -d --name my-registry --restart always -p 5000:5000 registry:2  
32df328a859903fa101053ecd30e081fb9935d1e3d09d71c88c7d8fe1a53f222
```

```
~ →
```

QUESTION 7 OF 10

Now its time to push some images to our registry server. Let's push two images for now i.e. `nginx:latest` and `httpd:latest`.

Note: Don't forget to pull them first.

To check the list of images pushed , use `curl -X GET localhost:5000/v2/_catalog`

Check 

Next →



Image nginx:latest pushed?



Image httpd:latest pushed?

[← Previous](#)[Next →](#)

```
~ → docker pull nginx:latest
docker pull httpd:latest
```

```
latest: Pulling from library/nginx
Digest: sha256:5aca99593157f4ae539a5dec1092a0ad8762f8e2eb1789085a13a0f5622369f6
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest
latest: Pulling from library/httpd
5b4d6ff92fc4: Already exists
f086d7c0b862: Pull complete
4f4fb700ef54: Pull complete
f60b6f23733a: Pull complete
c3f994ae0cac: Pull complete
ed804d184361: Pull complete
Digest: sha256:2a7deaaaf357261a1dffcb8fc725f4aaba2af95fbde1a40a68bca7cb0f03594e
Status: Downloaded newer image for httpd:latest
docker.io/library/httpd:latest
```

```
~ → docker tag nginx:latest localhost:5000/nginx:latest
docker tag httpd:latest localhost:5000/httpd:latest
```

```
~ → docker push localhost:5000/nginx:latest
docker push localhost:5000/httpd:latest
```

```
The push refers to repository [localhost:5000/nginx]
271182c95b03: Pushed
```

QUESTION 8 OF 10

Preview only — answers can't be changed or retried here.

Let's remove all the dangling images we have locally. Use `docker image prune -a` to remove them. How many images do we have now?

Note : Make sure we don't have any running containers except our `registry-sever`.

To get list of images use: `docker image ls`

3

4

~ → `docker images`

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx	latest	7aaca76c508f	4 days ago	161MB
localhost:5000/nginx	latest	7aaca76c508f	4 days ago	161MB
httpd	latest	b9097c67d4a2	7 days ago	117MB
localhost:5000/httpd	latest	b9097c67d4a2	7 days ago	117MB
mysql	latest	f6b0ca07d79d	7 months ago	934MB
postgres	latest	a38f9f77ff88	7 months ago	456MB
alpine	latest	706db57fb206	7 months ago	8.32MB
redis	latest	466e5b1da2ef	7 months ago	137MB
ubuntu	latest	97bed23a3497	7 months ago	78.1MB
registry	2	26b2eb03618e	2 years ago	25.4MB
kodekloud/simple-webapp-mysql	latest	129dd9f67367	7 years ago	96.6MB
kodekloud/simple-webapp	latest	c6e3cd9aae36	7 years ago	84.8MB

~ → `docker image prune -a`

WARNING! This will remove all images without at least one container associated to them.

Are you sure you want to continue? [y/N] y

Deleted Images:

untagged: kodekloud/simple-webapp:latest

untagged: kodekloud/simple-webapp@sha256:e5355b4c7804f453d79de75d6659ee702eeebbe30c02d9f1ce6602a96b576e57

deleted: sha256:c6e3cd9aae3645a98dd69c15b048614603efce6cda26c60f5f7e867ef68f729f

deleted: sha256:33833b97952fc68d999bc3bccaad23687ea6a939724e0130c151dc973ba8f2d3

deleted: sha256:a3dd002bb33a1cbb83aface983ea0d268be1b4ffda0e42ce72aa5c22ced6701f



Task

Hint

Solution

AI Assistant

38:46

QUESTION 10 OF 10

Let's clean up after ourselves.

Stop and remove the `my-registry` container.

Check



Next →



my-registry container removed?

Terminal 1

+

Stop Lab

⌵

🗒

⋮

```
~ → docker pull localhost:5000/nginx
```

```
Using default tag: latest
```

```
latest: Pulling from nginx
```

```
8211d5969d2e: Pull complete
```

```
830625e1ac85: Pull complete
```

```
5431d0092ffd: Pull complete
```

```
b4a248c845e5: Pull complete
```

```
7f8b1a2b17d8: Pull complete
```

```
45381ecb0e2f: Pull complete
```

```
13fd728be9eb: Pull complete
```

```
Digest: sha256:ae33218af50e1d8613f388276f0f29aacdbb815da94cf85f30b13df070022bac
```

```
Status: Downloaded newer image for localhost:5000/nginx:latest
```

```
localhost:5000/nginx:latest
```

```
~ → docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
32df328a8599	registry:2	"/entrypoint.sh /etc..."	11 minutes ago	Up 11 minutes	0.0.0.0:5000->5000/tcp
	my-registry				

```
~ → docker stop 32 && docker rm 32
```

```
32
```

```
32
```

```
~ →
```




QUESTION 1 OF 9



Preview only — answers can't be changed or retried here.

Explore the current setup and identify the number of networks that exist on this system.

1

4

3



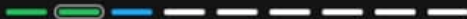
2

```
~ → docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
3cf7221ecd61	bridge	bridge	local
c9ae16eb7e15	host	host	local
b90d714f40ac	none	null	local

```
~ →
```

QUESTION 2 OF 9



Preview only — answers can't be changed or retried here.

What is the ID associated with the bridge network?

d6d852f6099426dadfdad90852f13b9a8695e93d20f0
28de939baf2c047faf85

fb9c1a74a36514f6f2d767eee7be024a0ffd40b6cf2088
86de42ee8003f5a6a6

3cf7221ecd6118901157d873e858071e74ea4f33ccb92
afd32fb81fb355aad1a



~ → docker network ls

NETWORK ID	NAME	DRIVER	SCOPE
3cf7221ecd61	bridge	bridge	local
c9ae16eb7e15	host	host	local
b90d714f40ac	none	null	local

~ →

QUESTION 3 OF 9

Preview only — answers can't be changed or retried here.

We just ran a container named `alpine-1`. Identify the network it is attached to.

none

bridge

host

Correct Answer

```
~ → docker inspect alpine-1 | grep -i net
      "NetworkMode": "host",
      "NetworkSettings": {
        "SandboxKey": "/var/run/docker/netns/default",
        "Networks": {
          "NetworkID": "c9ae16eb7e1591f4393fb7fed794bc6088832c131003d84372b909d880bd1c0e",
```

```
~ → docker network ls
NETWORK ID      NAME      DRIVER      SCOPE
3cf7221ecd61    bridge    bridge       local
c9ae16eb7e15    host      host         local
b90d714f40ac    none      null         local
```

```
~ →
```

QUESTION 4 OF 9



Preview only — answers can't be changed or retried here.

What is the subnet configured on `bridge` network?

172.12.0.0/24



192.168.0.1/24

182.18.0.1/16



Correct Answer

```
~ → docker network inspect bridge | grep -i subnet  
      "Subnet": "172.12.0.0/24",
```

```
~ →
```

QUESTION 5 OF 9



Run a container named `alpine-2` using the `alpine` image and attach it to the `none` network.

Check



Next →



Name: alpine-2 attached to none network

```
~ → docker run --name alpine-2 --network none alpine
```

```
~ →
```



QUESTION 6 OF 9

Create a new network named `wp-mysql-network` using the `bridge` driver. Allocate subnet `182.18.0.0/24`. Configure Gateway `182.18.0.1`

Check 

Next →

- ✓ Name: wp-mysql-network
- ✓ Driver: bridge
- ✓ subnet: 182.18.0.0/24
- ✓ Gateway: 182.18.0.1

```
~ → docker network create --driver bridge --subnet 182.18.0.0/24 --gateway 182.18.0.1 wp-mysql-network  
deb6ff8acdb62424009b928f62a15726d9732aec36eddddef8b7d802159229197
```

```
~ → docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
3cf7221ecd61	bridge	bridge	local
c9ae16eb7e15	host	host	local
b90d714f40ac	none	null	local
deb6ff8acdb6	wp-mysql-network	bridge	local

```
~ →
```

QUESTION 7 OF 9

Deploy a `mysql` database using the `mysql:5.7` image and name it `mysql-db`. Attach it to the newly created network `wp-mysql-network`.

Set the database password to use `db_pass123`. The environment variable to set is `MYSQL_ROOT_PASSWORD`.

Check

Next →

- ✓ Name: `mysql-db`
- ✓ Image: `mysql:5.7`
- ✓ Env: `MYSQL_ROOT_PASSWORD=db_pass123`
- ✓ Network: `wp-mysql-network`

```
~ → docker run -d -e MYSQL_ROOT_PASSWORD=db_pass123 --name mysql-db --network wp-mysql-network mysql:5.7
Unable to find image 'mysql:5.7' locally
5.7: Pulling from library/mysql
20e4dcae4c69: Pull complete
1c56c3d4ce74: Pull complete
e9f03a1c24ce: Pull complete
68c3898c2015: Pull complete
6b95a940e7b6: Pull complete
90986bb8de6e: Pull complete
ae71319cb779: Pull complete
ffc89e9dfd88: Pull complete
43d05e938198: Pull complete
064b2d298fba: Pull complete
df9a4d85569b: Pull complete
Digest: sha256:4bc6bc963e6d8443453676cae56536f4b8156d78bae03c0145cbe47c2aad73bb
Status: Downloaded newer image for mysql:5.7
d199159b0786e71b9a0da6f8315619177402bce8ec7a4b68fab0a9c267ddef1a
```

```
~ →
```


~ → docker ps

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
d199159b0786	mysql:5.7	"docker-entrypoint.s..."	3 minutes ago	Up 3 minutes	3306/tcp, 33060/tcp	mysql-db
aa5aee576fd4	alpine	"sleep 1000"	12 minutes ago	Up 12 minutes		alpine-1

```
~ → docker run --network=wp-mysql-network -e DB_Host=mysql-db -e DB_Password=db_pass123 -p 38080:8080 --name webapp --link mysql-db:mysql-db -d kodekloud/simple-webapp-mysql
e4618a84926db501c3aa13506376ec995f35d9a7b71b975b72d7ca35d53f5e99
```

~ → docker ps

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
e4618a84926d	kodekloud/simple-webapp-mysql	"/bin/sh -c 'python ..."	34 seconds ago	Up 33 seconds	0.0	webapp
.0.0:38080->8080/tcp, :::38080->8080/tcp						
d199159b0786	mysql:5.7	"docker-entrypoint.s..."	4 minutes ago	Up 4 minutes	3306/tcp, 33060/tcp	mysql-db
aa5aee576fd4	alpine	"sleep 1000"	14 minutes ago	Up 14 minutes		alpine-1

~ → █



SUCCESS

Successfully connected to the MySQL database.

Environment Variables: DB_Host=mysql-db; DB_Database=Not Set; DB_User=Not Set; DB_Password=db_pass123;

From e4618a84926d!